LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 2

ANDROID LAYOUT WITH COMPOSE

Oleh:

Arinda Dita Pratiwi NIM. 2410817120018

PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
APRIL 2026

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE
MODUL 2

Laporan Praktikum Pemrograman Mobile Modul 2: Android Layout with Compose ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Arinda Dita Pratiwi
NIM : 2410817120018

Menyetujui, Asisten Praktikum	Mengetahui, Dosen Penanggung Jawab Praktikum
Muhammad Erza Raihan NIM. 2310817210027	Irham Maulani Abdul Gani, S.Kom., M.Kom. NIP. 199710312025061009

DAFTAR ISI

LEMBAR PENGESAHAN 2
DAFTAR ISI 3
DAFTAR GAMBAR 4
DAFTAR TABEL 5
SOAL 1 6
A. Source Code 7
B. Output Program 16
C. Pembahasan 19
SOAL 2 24
A. Pembahasan 24
ADDITIONAL QUESTION 26
Daftar Pustaka 29
TAUTAN GIT 30

DAFTAR GAMBAR

Gambar 1. Tampilan Awal Aplikasi 6
Gambar 2. Tampilan Pilihan Persentase Tip..... 7
Gambar 3. Tampilan Aplikasi Setelah DIjalankan 7
Gambar 4. Screenshot Hasil Jawaban Soal 1 XML..... 16
Gambar 5. Screenshot Hasil Jawaban Soal 1 XML..... 16
Gambar 6. Screenshot Hasil Jawaban Soal 1 XML..... 17
Gambar 7. Screenshot Hasil Jawaban Soal 1 Compose..... 17
Gambar 8. Screenshot Hasil Jawaban Soal 1 Compose..... 18
Gambar 9. Screenshot Hasil Jawaban Soal 1 Compose..... 18

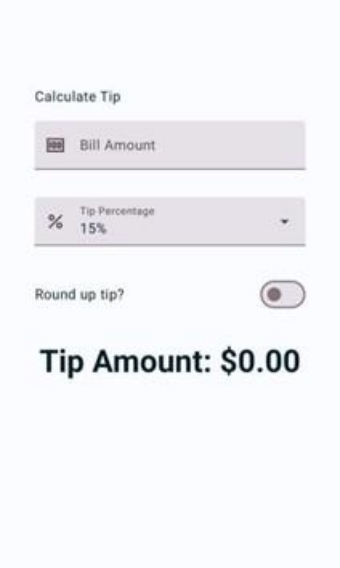
DAFTAR TABEL

Tabel 1. Source Code MainActivity.kt XML 7
Tabel 2. Source Code activity_main.xml..... 10
Tabel 3. Source Code string.xml..... 12
Tabel 4. Source Code MainActivity.kt Compose 12

SOAL 1

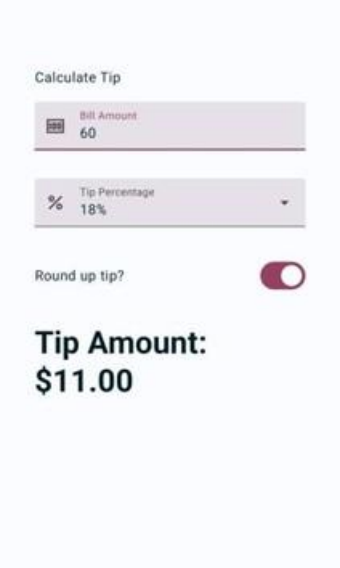
Buatlah sebuah aplikasi kalkulator tip menggunakan XML dan Jetpack Compose yang dirancang untuk membantu pengguna menghitung tip yang sesuai berdasarkan total biaya layanan yang mereka terima. Fitur-fitur yang diharapkan dalam aplikasi ini mencakup:

a. Input biaya layanan: Pengguna dapat memasukkan total biaya layanan yang diterima dalam bentuk nominal.
b. Pilihan persentase tip: Pengguna dapat memilih persentase tip yang diinginkan.
c. Pengaturan pembulatan tip: Pengguna dapat memilih untuk membulatkan tip ke angka yang lebih tinggi.
d. Tampilan hasil: Aplikasi akan menampilkan jumlah tip yang harus dibayar secara langsung setelah pengguna memberikan input.



Gambar 1. Tampilan Awal Aplikasi

Gambar 2. Tampilan Pilihan Persentase Tip



Gambar 3. Tampilan Aplikasi Setelah DIjalankan

A. Source Code

MainActivity.kt XML

Tabel 1. Source Code MainActivity.kt XML

```
package com.example.tipcalculatorxml

import android.content.res.ColorStateList
```

```
import android.graphics.Color
import android.os.Bundle
import android.text.Editable
import android.text.TextWatcher
import android.widget.ArrayAdapter
import android.widget.AutoCompleteTextView
import android.widget.EditText
import android.widget.TextView
import androidx.appcompat.app.AppCompatActivity
import com.google.android.material.materialswitch.MaterialSwitch
import java.text.NumberFormat
import java.util.*
import kotlin.math.ceil
class MainActivity : AppCompatActivity() {
    private lateinit var etBillAmount: EditText
    private lateinit var autoCompleteTip: AutoCompleteTextView
    private lateinit var switchRoundUp: MaterialSwitch
    private lateinit var tvTipAmount: TextView

    private val tipPercentages = listOf(15.0, 18.0, 20.0)
    private var selectedTipPercentage = 15.0

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
        etBillAmount = findViewById(R.id.etBillAmount)
        autoCompleteTip = findViewById(R.id.spinnerTipPercentage)
        switchRoundUp = findViewById(R.id.switchRoundUp)
        tvTipAmount = findViewById(R.id.tvTipAmount)

        setupDropdown()
        setupListeners()
        calculateTip()
    }

    private fun setupDropdown() {
        val items = arrayOf("15%", "18%", "20%")
        val adapter = ArrayAdapter(this, android.R.layout.simple_dropdown_item_1line, items)
        autoCompleteTip.setAdapter(adapter)
        autoCompleteTip.setOnItemClickListener { _, _, position, _ ->
            selectedTipPercentage = tipPercentages[position]
```

```
            calculateTip()
        }
    }

    private fun setupListeners() {
        etBillAmount.addTextChangedListener(object : TextWatcher {
            override fun afterTextChanged(s: Editable?) { calculateTip() }
            override fun beforeTextChanged(s: CharSequence?, start: Int, count: Int, after: Int) {}
            override fun onTextChanged(s: CharSequence?, start: Int, before: Int, count: Int) {}
        })

        switchRoundUp.trackTintList = ColorStateList.valueOf(Color.parseColor("#E0E0E0"))
        switchRoundUp.setOnCheckedChangeListener { _, isChecked ->
            if (isChecked) {
                switchRoundUp.trackTintList = ColorStateList.valueOf(Color.parseColor("#6750A4"))
            } else {
                switchRoundUp.trackTintList = ColorStateList.valueOf(Color.parseColor("#E0E0E0"))
            }
            calculateTip()
        }
    }

    private fun calculateTip() {
        val billAmountText = etBillAmount.text.toString()
        val billAmount = billAmountText.toDoubleOrNull() ?: 0.0
        var tip = billAmount * (selectedTipPercentage / 100.0)
        if (switchRoundUp.isChecked) {
            tip = ceil(tip)
        }

        val formattedTip = NumberFormat.getCurrencyInstance(Locale.US).format(tip)
        tvTipAmount.text = "Tip Amount: $formattedTip"
    }
}
```

activity_main.xml

Tabel 2. Source Code activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<ScrollView
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="#FAF7FA">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:padding="24dp">

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Calculate Tip"
            android:textSize="20sp"
            android:textColor="#49454F"
            android:layout_marginBottom="20dp" />

<com.google.android.material.textfield.TextInputLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginBottom="16dp"
            app:boxBackgroundColor="#E9E0EB"
            app:hintTextColor="#6750A4"
       app:startIconDrawable="@drawable/baseline_money_24"
            app:startIconTint="#49454F"
style="@style/Widget.MaterialComponents.TextInputLayout.Fi
lledBox">

<com.google.android.material.textfield.TextInputEditText
                android:id="@+id/etBillAmount"
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:hint="Bill Amount"
                android:inputType="numberDecimal"
                android:textColor="#1C1B1F"
                android:textSize="16sp" />
</com.google.android.material.textfield.TextInputLayout>

```

```
<com.google.android.material.textfield.TextInputLayout
style="@style/Widget.MaterialComponents.TextInputLayout.FilledBox.ExposedDropdownMenu"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginBottom="32dp"
            app:boxBackgroundColor="#E9E0EB"
            app:hintTextColor="#6750A4"
app:startIconDrawable="@drawable/baseline_percent_24"
            app:startIconTint="#49454F">

            <AutoCompleteTextView
                android:id="@+id/spinnerTipPercentage"
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:hint="Tip Percentage"
                android:inputType="none"
                android:text="15%"
                android:textSize="16sp" />
</com.google.android.material.textfield.TextInputLayout>

        <RelativeLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginBottom="40dp">

            <TextView
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:text="Round up tip?"
                android:textSize="16sp"
                android:layout_centerVertical="true"
                android:textColor="#1C1B1F" />

<com.google.android.material.materialswitch.MaterialSwitch
                android:id="@+id/switchRoundUp"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:layout_alignParentEnd="true"
                android:layout_centerVertical="true"
                app:thumbTint="@android:color/white" />
        </RelativeLayout>

        <TextView
            android:id="@+id/tvTipAmount"
            android:layout_width="match_parent"
```

```
            android:layout_height="wrap_content"
            android:text="Tip Amount: $0.00"
            android:textAlignment="center"
            android:textSize="32sp"
            android:textStyle="bold"
            android:textColor="#49454F" />

    </LinearLayout>
</ScrollView>
```

string.xml

Tabel 3. Source Code string.xml

```
<resources>
    <string name="app_name">Tip Calculator</string>
    <string-array name="tip_percentages">
        <item>15%</item>
        <item>18%</item>
        <item>20%</item>
    </string-array>
</resources>
```

MainActivity.kt Compose

Tabel 4. Source Code MainActivity.kt Compose

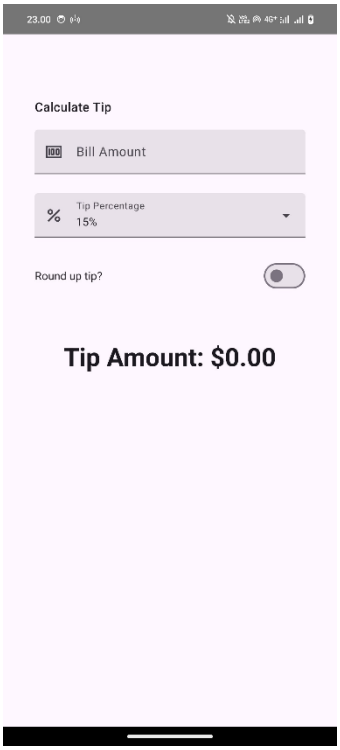
```
package com.example.tipcalculatorcompose

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.text.KeyboardOptions
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.Money
import androidx.compose.material.icons.filled.Percent
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.text.input.KeyboardType
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
```

```
import java.text.NumberFormat
import java.util.*

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            Surface(modifier = Modifier.fillMaxSize(), color = MaterialTheme.colorScheme.background) {
                TipCalculatorLayout()
            }
        }
    }
}

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun TipCalculatorLayout() {
    var amountInput by remember { mutableStateOf("") }
    var tipPercent by remember { mutableStateOf(15.0) }
    var roundUp by remember { mutableStateOf(false) }
    var expanded by remember { mutableStateOf(false) }

    val amount = amountInput.toDoubleOrNull() ?: 0.0
    val tipOptions = listOf(15.0, 18.0, 20.0)

    var tip = amount * (tipPercent / 100)
    if (roundUp) {
        tip = kotlin.math.ceil(tip)
    }
    val formattedTip = NumberFormat.getCurrencyInstance(Locale.US).format(tip)

    Column(
        modifier = Modifier
            .padding(horizontal = 40.dp, vertical = 80.dp)
            .fillMaxSize(),
        horizontalAlignment = Alignment.Start,
        verticalArrangement = Arrangement.Top
    ) {
        Text(
            text = "Calculate Tip",
            style = MaterialTheme.typography.titleMedium,
            modifier = Modifier.padding(bottom = 16.dp)
        )

        TextField(
```

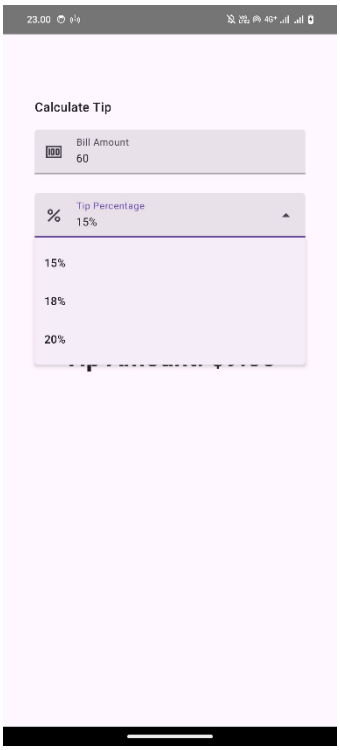
```
            value = amountInput,
            onValueChange = { amountInput = it },
            label = { Text("Bill Amount") },
            leadingIcon = { Icon(Icons.Default.Money, contentDescription = null) },
            modifier = Modifier.fillMaxWidth(),
            keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Number),
            singleLine = true
        )

        Spacer(modifier = Modifier.height(24.dp))

        ExposedDropdownMenuBox(
            expanded = expanded,
            onExpandedChange = { expanded = !expanded }
        ) {
            TextField(
                value = "${tipPercent.toInt()}%",
                onValueChange = {},
                readOnly = true,
                label = { Text("Tip Percentage") },
                leadingIcon = { Icon(Icons.Default.Percent, contentDescription = null) },
                trailingIcon = { ExposedDropdownMenuDefaults.TrailingIcon(expanded = expanded) },
                colors = ExposedDropdownMenuDefaults.textFieldColors(),
                modifier = Modifier.menuAnchor().fillMaxWidth()
            )
            ExposedDropdownMenu(
                expanded = expanded,
                onDismissRequest = { expanded = false }
            ) {
                tipOptions.forEach { option ->
                    DropdownMenuItem(
                        text = { Text("${option.toInt()}%") },
                        onClick = {
                            tipPercent = option
                            expanded = false
                        }
                    )
                }
            }
```

```
        }

        Spacer(modifier = Modifier.height(24.dp))

        Row(
            modifier = Modifier.fillMaxWidth(),
            verticalAlignment = Alignment.CenterVertically,
            horizontalArrangement = Arrangement.SpaceBetween
        ) {
            Text(text = "Round up tip?")
            Switch(
                checked = roundUp,
                onCheckedChange = { roundUp = it }
            )
        }

        Spacer(modifier = Modifier.height(60.dp))

        Box(
            modifier = Modifier.fillMaxWidth(),
            contentAlignment = Alignment.Center
        ) {
            Text(
                text = "Tip Amount: $formattedTip",
                style = MaterialTheme.typography.headlineMedium.copy(
                    fontWeight = FontWeight.Bold,
                    fontSize = 32.sp
                )
            )
        }
    }
}

```

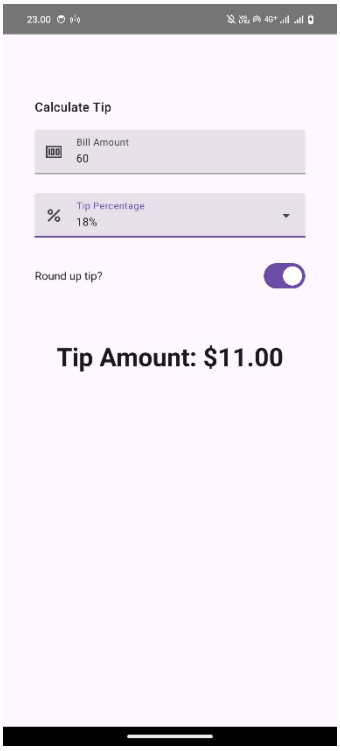
B. Output Program



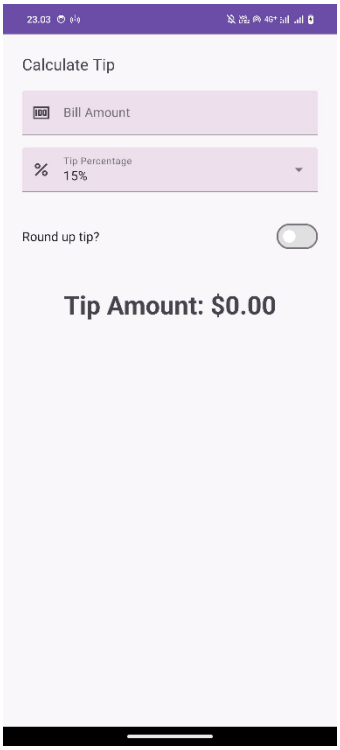
Gambar 4. Screenshot Hasil Jawaban Soal 1 XML



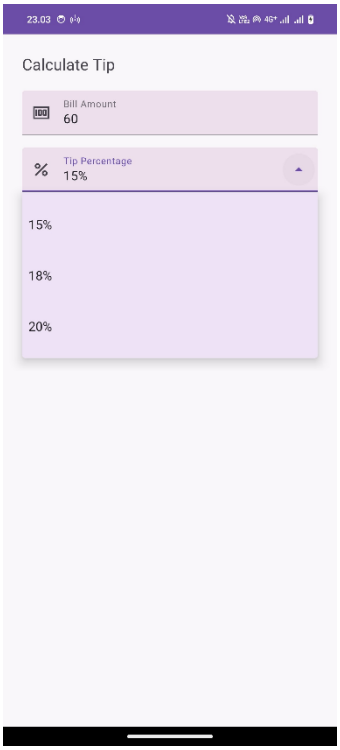
Gambar 5. Screenshot Hasil Jawaban Soal 1 XML



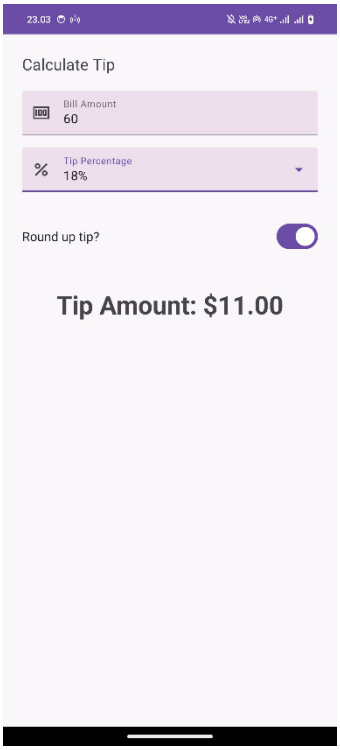
Gambar 6. Screenshot Hasil Jawaban Soal 1 XML



Gambar 7. Screenshot Hasil Jawaban Soal 1 Compose



Gambar 8. Screenshot Hasil Jawaban Soal 1 Compose



Gambar 9. Screenshot Hasil Jawaban Soal 1 Compose

C. Pembahasan

MainActivity.kt XML

File 'MainActivity.kt' merupakan pusat kendali aplikasi Tip Calculator berbasis XML. Class 'MainActivity' mewarisi 'AppCompatActivity', dan proses pertama yang terjadi di dalam fungsi 'onCreate()' adalah pemanggilan 'setContentView(R.layout.activity_main)' yang menghubungkan file Kotlin ini dengan tampilan yang sudah didefinisikan di 'activity_main.xml'. Setelah tampilan terhubung, empat komponen UI diambil menggunakan 'findViewById' dan disimpan ke dalam variabel yang telah dideklarasikan dengan 'lateinit var' di level class. 'etBillAmount' bertipe 'EditText' digunakan untuk menerima input nominal tagihan, 'autoCompleteTip' bertipe 'AutoCompleteTextView' sebagai dropdown pemilih persentase tip, 'switchRoundUp' bertipe 'MaterialSwitch' untuk mengaktifkan pembulatan, dan 'tvTipAmount' bertipe 'TextView' sebagai tempat menampilkan hasil perhitungan. Di luar fungsi 'onCreate()', terdapat dua properti tambahan di level class, yaitu 'tipPercentages' berupa list berisi nilai 15.0, 18.0, dan 20.0, serta 'selectedTipPercentage' yang menyimpan nilai tip yang sedang aktif dengan nilai awal 15.0.

Setelah semua komponen disiapkan, 'onCreate()' memanggil tiga fungsi sekaligus secara berurutan, yaitu 'setupDropdown()', 'setupListeners()', dan 'calculateTip()'. Pendekatan ini menjaga 'onCreate()' tetap ringkas dan setiap tanggung jawab dipisahkan ke fungsi masing-masing.

Fungsi 'setupDropdown()' bertanggung jawab menyiapkan pilihan persentase tip pada komponen dropdown. Di dalamnya, sebuah array string berisi "15%", "18%", dan "20%" dibungkus menggunakan 'ArrayAdapter' dengan layout bawaan Android 'android.R.layout.simple_dropdown_item_1line', lalu diserahkan ke 'autoCompleteTip' melalui 'setAdapter()'. Saat pengguna memilih salah satu item, listener 'setOnItemClickListener' menangkap posisi pilihan tersebut dan menggunakannya sebagai indeks untuk mengambil nilai numerik yang sesuai dari 'tipPercentages', kemudian disimpan ke 'selectedTipPercentage' dan langsung memicu 'calculateTip()'.

Fungsi 'setupListeners()' menangani dua sumber perubahan input. Pertama, 'etBillAmount' dipasang 'TextWatcher' melalui 'addTextChangedListener'. Dari tiga method yang wajib diimplementasikan, hanya 'afterTextChanged()' yang diisi logika, yaitu memanggil 'calculateTip()'

setiap kali teks berubah — dua method lainnya dibiarkan kosong karena tidak diperlukan. Kedua, 'switchRoundUp' dikonfigurasi warna track-nya secara manual menggunakan 'ColorStateList.valueOf()'. Saat switch dalam keadaan nonaktif, warna track diset ke abu-abu '#E0E0E0', dan ketika diaktifkan berubah menjadi ungu '#6750A4' yang selaras dengan tema Material You. Perubahan warna ini ditangani di dalam 'setOnCheckedChangeListener' yang sekaligus memanggil 'calculateTip()' setiap kali status switch berubah.

Seluruh perhitungan tip terpusat di fungsi 'calculateTip()'. Pertama, teks dari 'etBillAmount' diambil dan dikonversi ke 'Double' menggunakan 'toDoubleOrNull()' — jika konversi gagal karena input kosong atau bukan angka, nilai default 0.0 langsung digunakan sehingga aplikasi tidak crash. Nilai tip dihitung dengan mengalikan 'billAmount' dengan 'selectedTipPercentage / 100.0'. Jika 'switchRoundUp' dalam keadaan aktif, fungsi 'ceil()' dari paket 'kotlin.math' digunakan untuk membulatkan nilai tip ke atas ke angka bulat terdekat. Hasil akhirnya diformat menggunakan 'NumberFormat.getCurrencyInstance(Locale.US)' yang secara otomatis menambahkan simbol dolar dan pemisah ribuan, lalu ditampilkan ke 'tvTipAmount' sebagai string "Tip Amount: $formattedTip".

activity_main.xml

File 'activity_main.xml' mendefinisikan seluruh struktur tampilan aplikasi Tip Calculator. Root layout yang digunakan adalah 'ScrollView' dengan latar belakang berwarna krem muda '#FAF7FA', yang memastikan seluruh konten tetap dapat digulir pada layar dengan ukuran kecil. Di dalam 'ScrollView', terdapat satu 'LinearLayout' berorientasi vertikal dengan padding '24dp' di seluruh sisinya sebagai wadah utama yang menampung semua elemen UI secara berurutan dari atas ke bawah.

Elemen pertama adalah 'TextView' tanpa ID yang menampilkan judul "Calculate Tip" dengan ukuran teks '20sp' dan warna '#49454F', berfungsi murni sebagai label visual di bagian atas layar dengan jarak bawah '20dp' sebelum input pertama muncul.

Di bawah judul, terdapat komponen input tagihan yang dibungkus menggunakan 'TextInputLayout' dengan style 'Widget.MaterialComponents.TextInputLayout.FilledBox'. Pembungkus ini memberikan tampilan kotak berisi dengan warna latar '#E9E0EB' dan hint

berwarna ungu '#6750A4' sesuai tema Material You. Atribut 'app:startIconDrawable' menyisipkan ikon uang di sisi kiri kolom input, dengan tint '#49454F' agar warnanya selaras dengan teks judul. Di dalamnya terdapat 'TextInputEditText' dengan ID 'etBillAmount', dikonfigurasi dengan 'android:inputType="numberDecimal"' sehingga keyboard yang muncul secara otomatis adalah keyboard angka dengan dukungan desimal.

Komponen kedua adalah dropdown pemilih persentase tip, juga menggunakan 'TextInputLayout' namun dengan style 'FilledBox.ExposedDropdownMenu' yang secara khusus dirancang untuk menampilkan pilihan seperti spinner. Konfigurasi warna dan ikonnya serupa dengan input sebelumnya, hanya ikonnya diganti dengan 'baseline_percent_24'. Di dalamnya terdapat 'AutoCompleteTextView' dengan ID 'spinnerTipPercentage' yang dikonfigurasi dengan 'android:inputType="none"' untuk mencegah munculnya keyboard saat komponen ini disentuh, serta nilai awal "15%" langsung diset melalui 'android:text'.

Untuk baris switch pembulatan, digunakan 'RelativeLayout' sebagai wadah agar dua elemen dapat ditempatkan di sisi berlawanan dalam satu baris secara efisien. 'TextView' bertuliskan "Round up tip?" diposisikan di sisi kiri dan ditengahkan secara vertikal menggunakan 'android:layout_centerVertical="true"', sementara 'MaterialSwitch' dengan ID 'switchRoundUp' ditempatkan di sisi kanan menggunakan 'android:layout_alignParentEnd="true"' dengan posisi vertikal yang sama. Atribut 'app:thumbTint' pada switch diset ke putih agar bulatan kecil di atas track selalu berwarna putih bersih, sementara warna track-nya dikendalikan secara dinamis dari sisi Kotlin.

Elemen terakhir adalah 'TextView' dengan ID 'tvTipAmount' yang menampilkan hasil perhitungan tip. Teks awalnya diset ke "Tip Amount: $0.00" sebagai nilai default yang langsung terlihat saat aplikasi pertama dibuka. Ukuran teksnya '32sp' dengan style bold dan alignment center menjadikan komponen ini sebagai elemen paling menonjol secara visual di seluruh layar, sekaligus sebagai titik fokus utama yang langsung menarik perhatian pengguna.

strings.xml

File 'strings.xml' merupakan file sumber daya teks yang berada di direktori 'res/values/' dan berfungsi sebagai tempat pemusatan seluruh konten string yang digunakan dalam aplikasi.

Pendekatan ini merupakan praktik terbaik dalam pengembangan Android karena memisahkan konten teks dari logika kode maupun struktur layout, sehingga apabila ada perubahan teks atau penambahan dukungan bahasa baru, cukup dilakukan di satu tempat tanpa menyentuh file lainnya.

Di dalam file ini terdapat dua sumber daya. Pertama adalah 'string' dengan nama 'app_name' yang menyimpan nilai "Tip Calculator" sebagai nama resmi aplikasi yang ditampilkan di launcher maupun toolbar. Kedua adalah 'string-array' dengan nama 'tip_percentages' yang menampung tiga item secara berurutan, yaitu "15%", "18%", dan "20%". Meskipun pada implementasi aktual di 'MainActivity.kt' array pilihan tip didefinisikan langsung di dalam kode menggunakan 'arrayOf("15%", "18%", "20%")', keberadaan 'string-array' di sini mencerminkan pendekatan yang lebih ideal karena data statis seperti daftar pilihan sebaiknya tidak dikodekan keras di dalam logika program, melainkan dikelola sebagai sumber daya yang dapat diakses melalui 'R.array.tip_percentages' kapan pun dibutuhkan.

MainActivity.kt Compose

File 'MainActivity.kt' pada versi Jetpack Compose memiliki struktur yang jauh lebih ringkas dibandingkan versi XML. Class 'MainActivity' kini mewarisi 'ComponentActivity' sebagai ganti 'AppCompatActivity', dan di dalam 'onCreate()' tidak lagi terdapat pemanggilan 'setContentView()' maupun 'findViewById()'. Sebagai gantinya, fungsi 'setContent {}' digunakan untuk mendefinisikan tampilan langsung menggunakan kode Kotlin. Di dalamnya, 'Surface' dengan modifier 'fillMaxSize()' dan warna latar dari 'MaterialTheme.colorScheme.background' dijadikan wadah utama yang membungkus composable 'TipCalculatorLayout()' — seluruh logika dan tampilan aplikasi ada di dalam fungsi tersebut.

Fungsi 'TipCalculatorLayout()' dianotasi dengan '@Composable' dan '@OptIn(ExperimentalMaterial3Api::class)' karena menggunakan komponen 'ExposedDropdownMenuBox' yang masih berstatus eksperimental. Di bagian atas fungsi, seluruh state aplikasi dideklarasikan menggunakan 'remember' dan 'mutableStateOf'. 'amountInput' menyimpan teks input tagihan, 'tipPercent' menyimpan persentase tip yang aktif dengan nilai awal 15.0, 'roundUp' menyimpan status switch pembulatan, dan 'expanded' mengontrol apakah dropdown sedang terbuka atau tertutup. Berbeda dari versi XML yang menyimpan state di level

class, di Compose seluruh state hidup di dalam fungsi composable itu sendiri dan Compose akan secara otomatis me-recompose tampilan setiap kali nilai state berubah.

Perhitungan tip juga dilakukan langsung di dalam fungsi ini, bukan di fungsi terpisah. 'amountInput' dikonversi ke 'Double' menggunakan 'toDoubleOrNull()' dengan fallback 0.0, lalu tip dihitung dari perkalian amount dengan 'tipPercent / 100'. Jika 'roundUp' bernilai true, 'kotlin.math.ceil()' digunakan untuk membulatkan hasilnya. Nilai akhir langsung diformat menggunakan 'NumberFormat.getCurrencyInstance(Locale.US)'. Karena semua ini ditulis di luar blok UI secara deklaratif, perhitungan akan otomatis dijalankan ulang setiap kali state yang terlibat berubah — tanpa perlu memanggil fungsi hitung secara manual seperti di versi XML.

Struktur UI dibangun menggunakan 'Column' dengan padding horizontal '40.dp' dan vertikal '80.dp', dengan alignment horizontal ke kiri dan vertikal ke atas. Elemen pertama adalah 'Text' bertuliskan "Calculate Tip" yang menggunakan 'MaterialTheme.typography.titleMedium' sebagai style-nya. Di bawahnya terdapat 'TextField' untuk input tagihan yang dilengkapi 'leadingIcon' berupa 'Icons.Default.Money', dikonfigurasi dengan 'KeyboardOptions(keyboardType = KeyboardType.Number)' agar keyboard angka muncul otomatis, serta 'singleLine = true' untuk membatasi input hanya satu baris.

Untuk dropdown persentase tip, digunakan kombinasi 'ExposedDropdownMenuBox', 'TextField' mode readonly, dan 'ExposedDropdownMenu'. 'TextField' di dalamnya menampilkan nilai aktif seperti "15%" dan bersifat readonly karena 'onValueChange' dikosongkan. 'ExposedDropdownMenuDefaults.TrailingIcon()' secara otomatis menampilkan ikon panah yang berubah arah sesuai status 'expanded'. Di dalam 'ExposedDropdownMenu', fungsi 'forEach' digunakan untuk mengiterasi 'tipOptions' dan menghasilkan 'DropdownMenuItem' untuk setiap pilihan — saat diklik, 'tipPercent' diperbarui dan 'expanded' ditutup kembali.

Baris switch pembulatan dibangun menggunakan 'Row' dengan 'horizontalArrangement = Arrangement.SpaceBetween' sehingga teks "Round up tip?" berada di sisi kiri dan 'Switch' berada di sisi kanan secara otomatis tanpa memerlukan 'RelativeLayout' seperti pada versi XML. Perubahan state switch langsung ditangani melalui lambda 'onCheckedChange = { roundUp = it }'. Di bagian paling bawah, 'Box' dengan 'contentAlignment = Alignment.Center' digunakan untuk menampilkan hasil tip di tengah layar menggunakan 'Text' dengan style 'headlineMedium' yang dimodifikasi menggunakan '.copy()' untuk menambahkan 'FontWeight.Bold' dan ukuran '32.sp'.

SOAL 2

Jelaskan perbedaan dari implementasi XML dan Jetpack Compose beserta kelebihan dan kekurangan masing-masing implementasi.

A. Pembahasan

Perbedaan paling mencolok terletak pada cara mendefinisikan UI. Pada XML, tampilan dipisah menjadi dua lapisan, yaitu struktur visual ditulis di 'activity_main.xml' dan logika ditulis di 'MainActivity.kt', lalu keduanya dihubungkan melalui 'setContentView()' dan 'findViewById()'. Pada Compose, tidak ada file XML sama sekali, di mana seluruh tampilan dan logika ditulis bersama dalam satu fungsi '@Composable' menggunakan Kotlin murni.

Perbedaan kedua adalah pengelolaan state. Di XML, tidak ada sistem state bawaan di mana developer harus secara manual membaca nilai komponen, memprosesnya, lalu menulis kembali hasilnya ke komponen lain. Pada contoh ini terlihat dari fungsi 'calculateTip()' yang harus dipanggil secara eksplisit setiap kali ada perubahan input. Di Compose, state dikelola melalui 'remember' dan 'mutableStateOf', sehingga setiap perubahan nilai secara otomatis memicu recomposition dan UI selalu mencerminkan kondisi terkini tanpa perlu dipanggil manual.

Perbedaan ketiga adalah cara menyusun layout. XML menggunakan hierarki tag seperti 'ScrollView', 'LinearLayout', dan 'RelativeLayout' yang bersifat deklaratif namun terpisah dari logika. Compose menggunakan fungsi layout seperti 'Column', 'Row', dan 'Box' yang langsung ditulis bersama logika dalam satu aliran kode Kotlin, sehingga kondisional seperti 'if (roundUp)' dapat langsung ditulis di tengah definisi UI.

XML unggul dalam hal kematangan ekosistem yang telah digunakan sejak awal Android sehingga dokumentasi, tutorial, dan dukungan komunitas sangat melimpah. Pemisahan antara layout dan logika juga memudahkan kolaborasi antara desainer dan developer karena file XML dapat diedit secara visual melalui Layout Editor di Android Studio. Namun kelemahannya, sinkronisasi antara dua lapisan ini rentan terhadap kesalahan, misalnya ID yang salah ketik di 'findViewById()' baru diketahui saat runtime, bukan saat kompilasi. Selain itu, untuk UI yang

kompleks dan dinamis, kode XML cenderung menjadi panjang dan sulit dikelola karena banyaknya tag bersarang.

Jetpack Compose unggul dalam produktivitas dan keamanan tipe karena seluruh UI ditulis dalam Kotlin, kesalahan seperti referensi komponen yang salah langsung terdeteksi saat kompilasi. Sistem state reaktifnya menghilangkan kebutuhan untuk memanggil fungsi pembaruan UI secara manual, sehingga kode lebih ringkas dan alur data lebih mudah dipahami. Compose juga mendukung preview langsung di Android Studio melalui anotasi '@Preview' tanpa harus menjalankan aplikasi ke perangkat. Kelemahannya, Compose masih relatif baru sehingga beberapa komponen masih berstatus eksperimental seperti 'ExposedDropdownMenuBox' yang memerlukan anotasi '@OptIn'. Kurva belajarnya juga lebih curam bagi developer yang sudah terbiasa dengan paradigma imperatif XML, karena Compose menuntut pemahaman tentang konsep reaktif dan recomposition yang berbeda secara fundamental.

.

ADDITIONAL QUESTION

1. Apa saja fitur yang menjadi daya jual pada Kotlin? (contoh: null safety, chain, dll) **

Jawab:

Salah satu fitur paling menonjol dari Kotlin adalah null safety. Dalam Java, NullPointerException (NPE) menjadi salah satu penyebab crash paling umum. Kotlin mengatasinya dengan membangun sistem null safety langsung ke dalam tipe data — secara default semua variabel tidak boleh null, dan jika diperlukan harus dideklarasikan eksplisit menggunakan tanda tanya ('?'). Ardito dkk. [2] membuktikan bahwa tim yang menggunakan Kotlin mengalami kejadian NPE yang jauh lebih sedikit dibandingkan tim Java, dan perbedaan ini signifikan secara statistik.

Fitur berikutnya adalah conciseness. Jemerov dan Isakova [1] menjelaskan bahwa fitur seperti data class, type inference, dan lambda expression memungkinkan kode yang jauh lebih pendek untuk tujuan yang sama. Ardito dkk. [2] memperkuat hal ini dengan temuan bahwa pengembang Kotlin menulis rata-rata tiga kali lebih sedikit baris kode dibandingkan Java untuk fitur yang identik.

Kotlin juga memperkenalkan smart cast dan safe cast operator ('as?'). Berbeda dari Java yang mengharuskan cast eksplisit dan berisiko 'ClassCastException', Kotlin secara otomatis mengenali tipe data setelah pengecekan sehingga pengembang tidak perlu menulis cast berulang [2].

Fitur default dan named arguments menyelesaikan masalah umum di Java, di mana semua argumen harus diisi meski nilainya selalu sama, sehingga memaksa pembuatan banyak versi method yang hampir identik. Kotlin memperbolehkan nilai default pada setiap parameter dan pemanggilan berdasarkan nama, membuat kode lebih fleksibel [1].

Extension functions memungkinkan penambahan fungsi baru ke class yang sudah ada tanpa pewarisan. Ardito dkk. [2] menyebut fitur ini sangat berguna dalam pengembangan Android karena pengembang dapat memperluas fungsionalitas library eksternal tanpa mengubah kode aslinya.

Terakhir, coroutines menyederhanakan pemrograman asinkron. Jemerov dan Isakova

[1] menjelaskan bahwa operasi latar belakang seperti pengambilan data dari internet atau akses database dapat ditulis jauh lebih bersih dibandingkan threads tradisional Java, tanpa mengorbankan performa.

Secara keseluruhan, fitur-fitur Kotlin bukan sekadar tambahan kosmetik, melainkan solusi nyata atas permasalahan Java. Kombinasi null safety, keringkasan kode, smart cast, default arguments, extension functions, dan coroutines menjadikan Kotlin bahasa yang lebih aman sekaligus lebih produktif untuk pengembangan aplikasi modern.

2. Berikan 1 test case terkait penggunaan fitur unggulan yang ada pada Kotlin (contoh: penggunaan null safety).

Jawab:

```
1   package com.example.nullsafetytest
2
3   import org.junit.Assert.assertEquals
4   import org.junit.Test
5
6   class NilaiMahasiswaTest {
7
8       @Test
9       fun testNilaiKosong() {
10          val input = ""
11          val nilai = input.toDoubleOrNull() ?: 0.0
12          assertEquals(0.0, nilai, 0.0)
13      }
14
15      @Test
16      fun testNilaiHuruf() {
17          val input = "abc"
18          val nilai = input.toDoubleOrNull() ?: 0.0
19          assertEquals(0.0, nilai, 0.0)
20      }
21
22      @Test
23      fun testNilaiValid() {
24          val input = "85"
25          val nilai = input.toDoubleOrNull() ?: 0.0
26          assertEquals(85.0, nilai, 0.0)
27      }
```

```
28
29      @Test
30      fun testNilaiDiatasSeratus() {
31          val input = "110"
32          val nilai = input.toDoubleOrNull() ?: 0.0
33          val hasil = if (nilai > 100) 100.0 else nilai
34          assertEquals(100.0, hasil, 0.0)
35      }
36  }
```

File 'NilaiMahasiswaTest.kt' berisi empat test case yang menguji fitur null safety Kotlin menggunakan tema input nilai mahasiswa. Inti pengujiannya ada pada 'toDoubleOrNull()' dikombinasikan dengan operator Elvis ('?:'). Jika input tidak bisa dikonversi ke angka maka nilainya otomatis diganti 0.0 tanpa menyebabkan crash.

'testNilaiKosong()' mensimulasikan kondisi field nilai dibiarkan kosong. Input berupa string kosong tidak bisa dikonversi ke Double, sehingga 'toDoubleOrNull()' mengembalikan null dan operator Elvis menggantinya dengan 0.0.

'testNilaiHuruf()' mensimulasikan kesalahan ketik seperti memasukkan huruf di kolom nilai. Hasilnya sama, konversi gagal, null diganti 0.0, program tetap berjalan normal.

'testNilaiValid()' mensimulasikan input yang benar yaitu "85". Konversi berhasil, 'toDoubleOrNull()' mengembalikan 85.0, dan operator Elvis tidak perlu bekerja.

'testNilaiDiatasSeratus()' menambahkan satu lapisan logika, jika nilai yang dimasukkan melebihi 100, maka dikembalikan ke batas maksimal 100.0 menggunakan ekspresi 'if-else' langsung di dalam variabel.

Test Case	Input	Expected	Actual	Status
testNilaiKosong	""	0.0	0.0	Pass
testNilaiHuruf	"abc"	0.0	0.0	Pass
testNilaiValid	"85"	85.0	85.0	Pass
testNilaiDiatasSeratus	"110"	100.0	100.0	Pass

Keempat test case berhasil dijalankan dengan hasil BUILD SUCCESSFUL. Ini membuktikan bahwa null safety pada Kotlin bekerja dengan baik yaitu input yang tidak valid tidak menyebabkan crash, melainkan ditangani otomatis oleh 'toDoubleOrNull() ?: 0.0' tanpa perlu pengecekan manual seperti di Java.

Daftar Pustaka

[1] D. Jemerov and S. Isakova, *Kotlin in Action*. Shelter Island, NY: Manning Publications, 2023.

[2] L. Ardito, R. Coppola, G. Malnati, and M. Torchiano, "Effectiveness of Kotlin vs. Java in Android App Development Tasks," *Information and Software Technology*, vol. 127, 2020, doi: 10.1016/j.infsof.2020.106374.

TAUTAN GIT

Berikut adalah tautan untuk *source code* yang telah dibuat.

https://github.com/arinn-lab/Module-Mobile